

AWS GUARDDUTY AUTOMATED THREAT DETECTION & RESPONSE PIPELINE

Full Detection Engineering Lifecycle — Built From Scratch

Oluwaseyi Michael Falode

Cybersecurity & Cloud Security Engineer | Toronto, ON

linkedin.com/in/oluwaseyi-falode | github.com/seyifalode-cmd

May 2026

EXECUTIVE SUMMARY

This document provides a complete end-to-end walkthrough of an AWS-native automated threat detection and response pipeline built entirely from scratch. The project demonstrates the full detection engineering lifecycle — from initial threat detection through automated remediation, audit logging, evidence storage, and real-time monitoring — with zero reliance on pre-built stacks or templates.

The pipeline solves a core detection engineering problem: threats get logged but nothing automatically happens. Every component was selected, configured, and wired together to ensure that MEDIUM and HIGH severity findings are automatically triaged, alerted, and remediated within seconds of detection.

The end-to-end test generated 391 sample GuardDuty findings. The pipeline processed all qualifying findings automatically — routing through EventBridge, triggering Lambda for automated remediation, and delivering full JSON alert payloads to the analyst inbox within seconds. Zero manual intervention.

ARCHITECTURE OVERVIEW

Eight AWS services configured manually from scratch, wired in sequence:

Component	Role in the Pipeline
GuardDuty	Detection — monitors API calls, network traffic, container behavior, IAM activity, DNS queries continuously.
EventBridge	Routing — severity filter (≥ 4) routes qualified findings to both SNS and Lambda simultaneously.
SNS	Alerting — delivers full JSON finding payload to analyst email the moment EventBridge fires.
Lambda	Remediation — Python function auto-isolates compromised EC2 on HIGH severity (≥ 7) findings.
IAM	Permissions — least privilege role giving Lambda exactly what it needs to operate.
CloudTrail	Audit trail — captures every API call for forensic investigation and compliance.

S3	Evidence storage — durable long-term storage for all CloudTrail logs.
CloudWatch	Monitoring — real-time dashboard tracking pipeline health and log ingestion.

Detection flow: GuardDuty → EventBridge (severity filter) → SNS (alert) + Lambda (remediate) → CloudTrail + S3 (log) → CloudWatch (monitor)

PHASE 1: GUARDDUTY — DETECTION LAYER

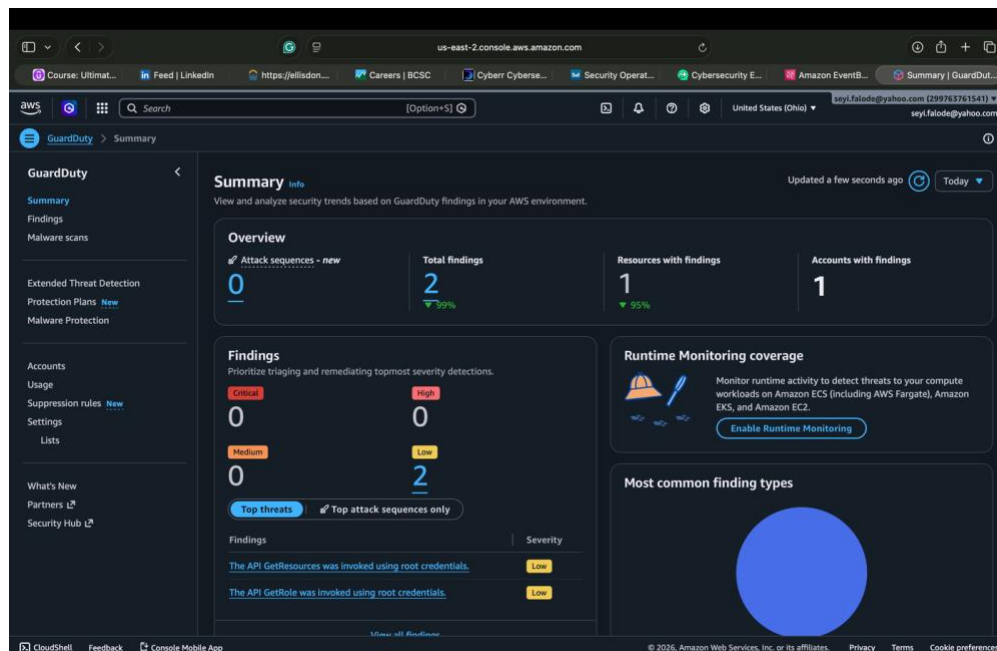
GuardDuty was enabled in the AWS account monitoring us-east-2 (Ohio). It serves as the primary detection engine — continuously analyzing multiple data sources for suspicious activity and generating structured findings with MITRE ATT&CK mappings.

What GuardDuty Monitors

- CloudTrail logs — every API call made in the account
- VPC Flow Logs — all network traffic in and out of resources
- DNS logs — domain name queries from EC2 instances
- EKS audit logs — Kubernetes API calls and container behavior
- Runtime monitoring — process-level activity inside containers and EC2

Severity Scale

- LOW (1-3) — informational, logged only, no automated action triggered
- MEDIUM (4-6) — notable, triggers full alert and response pipeline
- HIGH (7-10) — critical, triggers immediate automated EC2 isolation



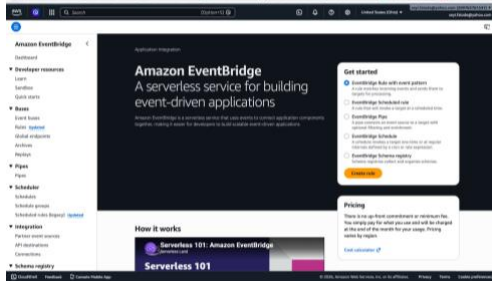
Screenshot 1 — GuardDuty Summary Dashboard showing active findings

PHASE 2: EVENTBRIDGE — ROUTING & AUTOMATION LAYER

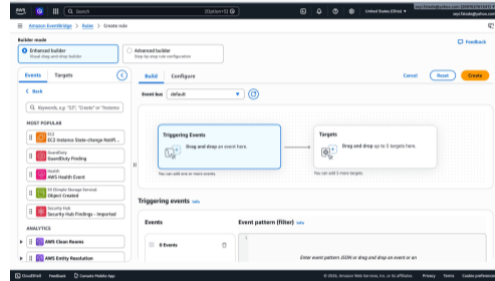
EventBridge is the central nervous system of the pipeline. It listens on the default event bus for GuardDuty findings and applies a severity filter before routing to downstream targets — preventing low-severity noise from flooding the response chain.

Rule Configuration

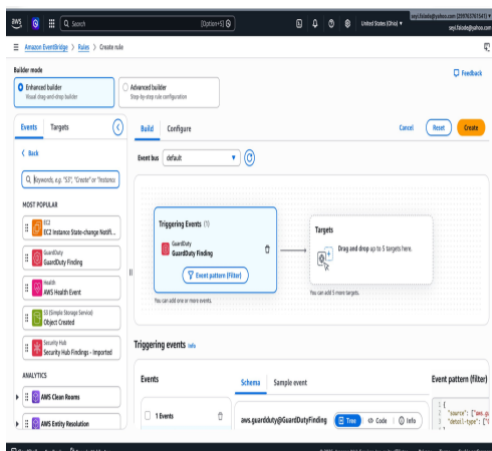
- Rule name: guardduty-high-severity-response
- Event bus: default | Status: Enabled
- Severity filter: { "severity": [{ "numeric": [">=", 4] }] } — only MEDIUM and HIGH findings trigger
- Target 1: guardduty-alerts (SNS) — email alert to analyst
- Target 2: guardduty-auto-response (Lambda) — automated remediation



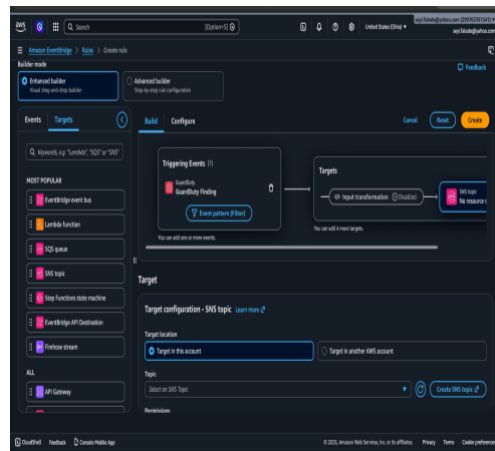
Screenshot 2 — EventBridge homepage



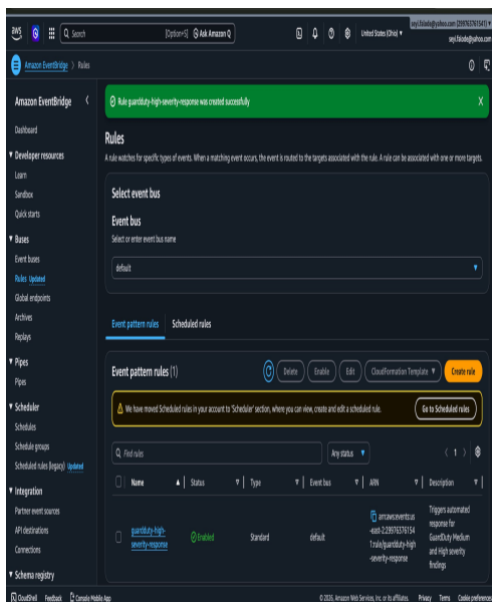
Screenshot 3 — GuardDuty Finding in event list



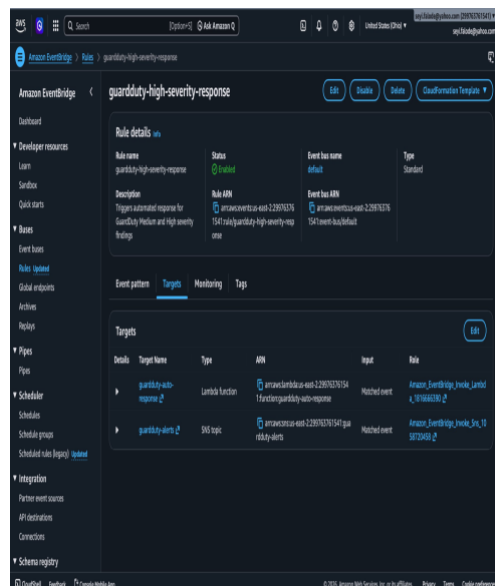
Screenshot 4 — GuardDuty Finding dragged to canvas



Screenshot 5 — SNS target added, target invalid until topic created



Screenshot 6 — Rule created and enabled



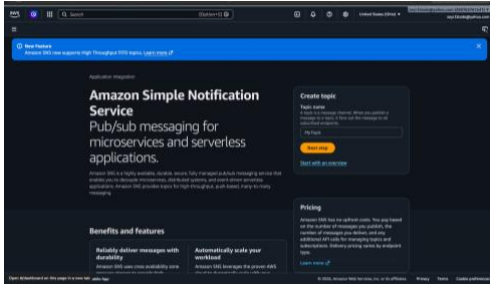
Screenshot 7 — Both Lambda and SNS targets confirmed

PHASE 3: SNS — ALERTING LAYER

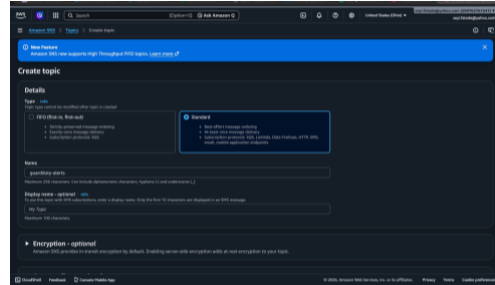
Amazon SNS handles the human-facing side of the pipeline. When EventBridge routes a qualifying finding, SNS immediately delivers the full JSON finding payload as an email to the subscribed analyst address.

Configuration

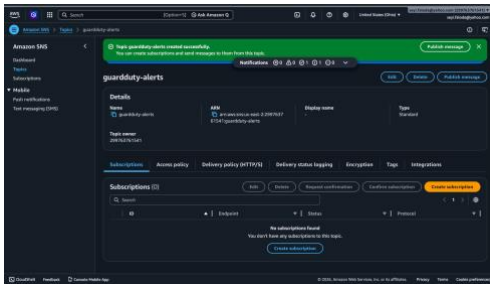
- Topic name: guardduty-alerts | Type: Standard | Region: us-east-2
- ARN: arn:aws:sns:us-east-2:299763761541:guardduty-alerts
- Subscription: Email → seyi.falode@yahoo.com | Status: Confirmed
- Note: Confirmation email landed in Yahoo Junk — whitelist AWS notification emails in production



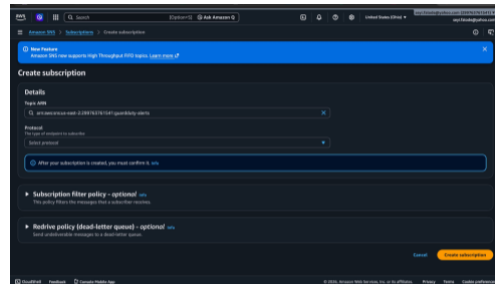
Screenshot 8 — SNS homepage



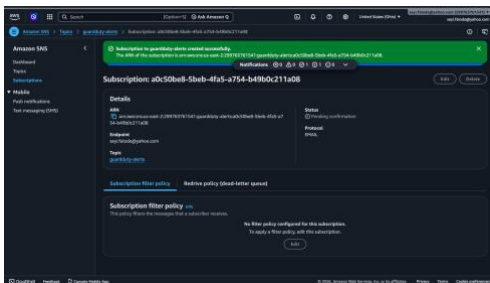
Screenshot 9 — Creating topic guardduty-alerts



Screenshot 10 — Topic created successfully



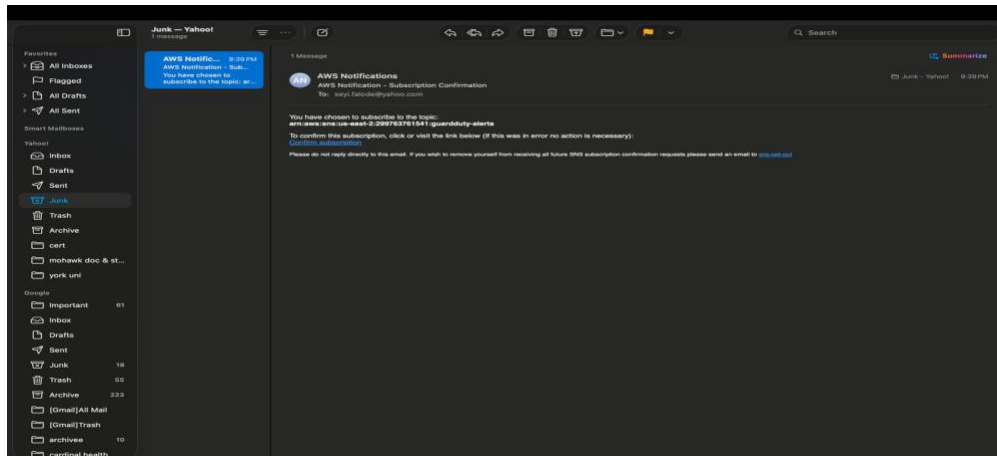
Screenshot 11 — Creating email subscription



Screenshot 12 — Subscription created, pending confirmation



Screenshot 13 — Subscription confirmed



Screenshot 14 — AWS confirmation email received in Yahoo (landed in Junk)

PHASE 4: LAMBDA — AUTOMATED REMEDIATION LAYER

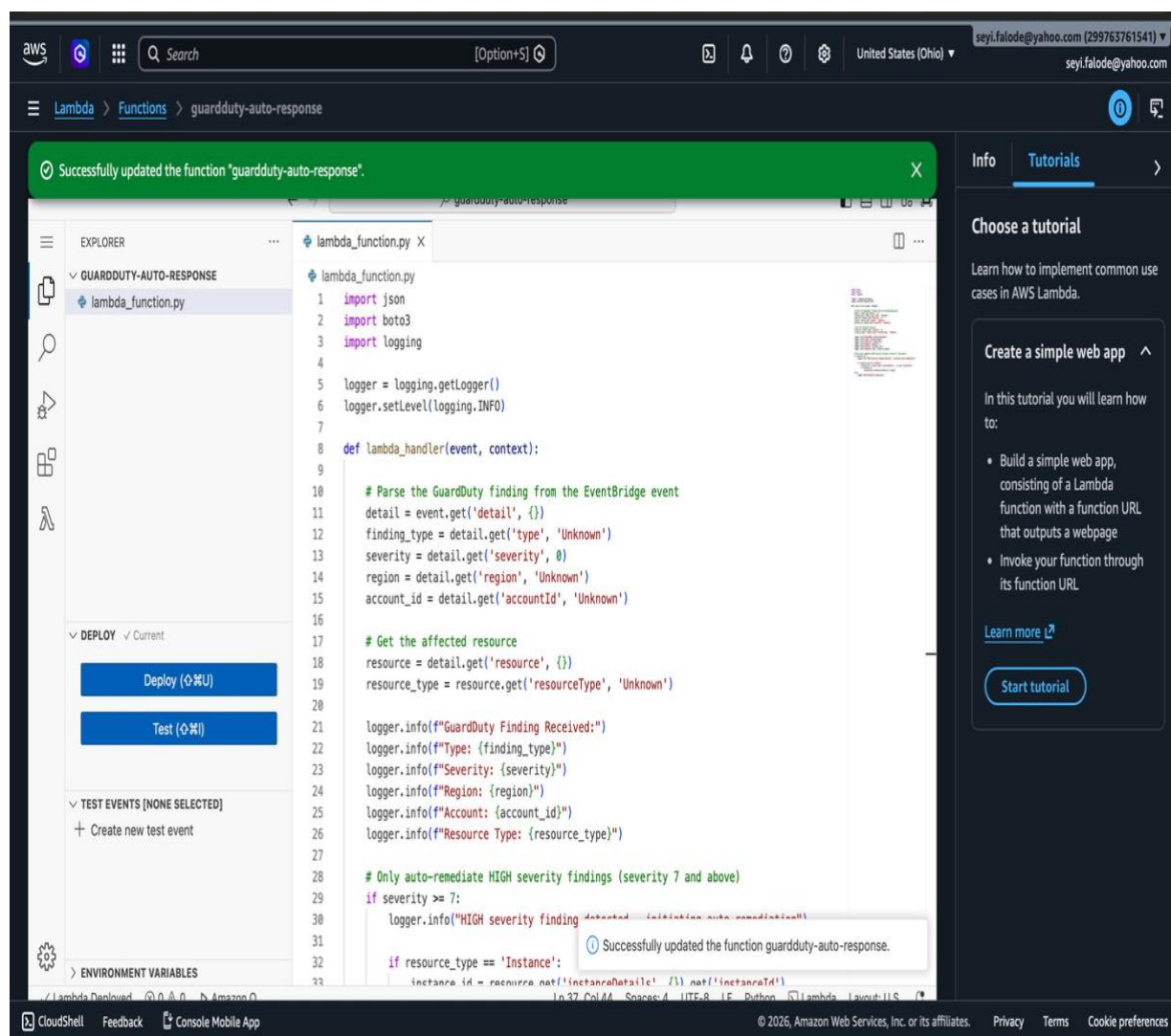
The Lambda function is the response engine. Written in Python 3.12, it receives GuardDuty findings from EventBridge, checks the severity, and either auto-remediates (HIGH) or logs and monitors (MEDIUM). This ensures no human needs to be watching a dashboard for HIGH threats to be acted on.

Function Details

- Function name: guardduty-auto-response | Runtime: Python 3.12 | Region: us-east-2
- Severity >= 7 (HIGH): Tags compromised EC2 as COMPROMISED and initiates isolation
- Severity 4-6 (MEDIUM): Logs finding to CloudWatch, no destructive action — avoids false positive disruption
- Severity < 4 (LOW): Not triggered — filtered out upstream by EventBridge

Key Code Logic

```
if severity >= 7: isolate_ec2_instance(instance_id, region) # tag as COMPROMISED + remove from rotation
```



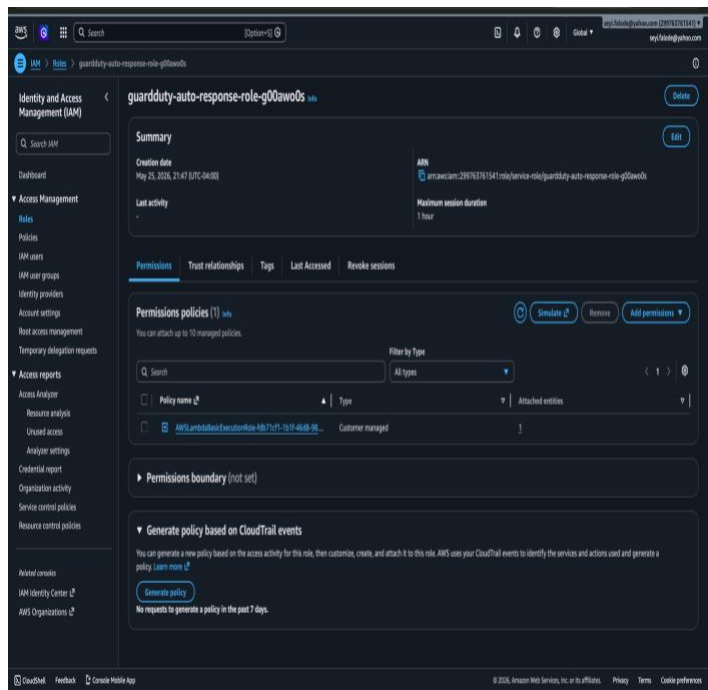
Screenshot 15 — Lambda function guardduty-auto-response deployed successfully

PHASE 5: IAM — LEAST PRIVILEGE PERMISSIONS

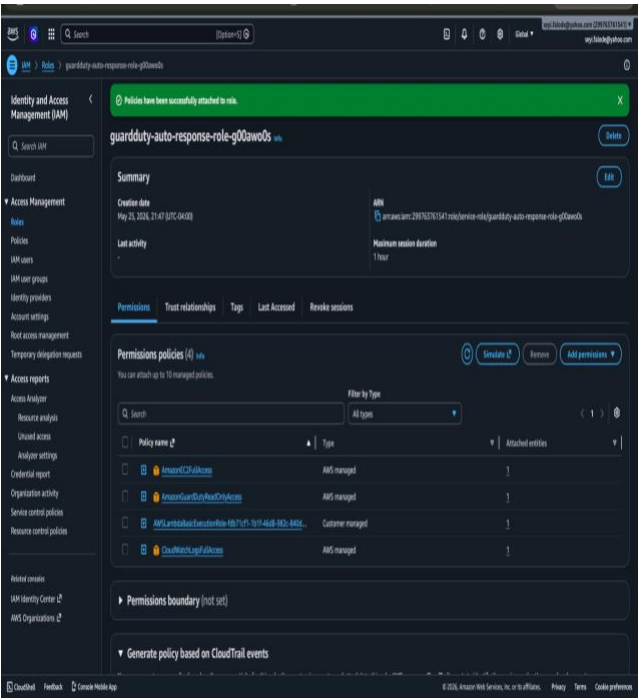
The Lambda execution role was configured with four policies — the minimum required for the function to operate. This follows the principle of least privilege: Lambda only has access to exactly what it needs.

Policies Attached

- AWSLambdaBasicExecutionRole — auto-created, base permission allowing Lambda to execute
- AmazonEC2FullAccess — allows Lambda to tag and modify EC2 instances during isolation
- AmazonGuardDutyReadOnlyAccess — allows Lambda to read finding details from GuardDuty
- CloudWatchLogsFullAccess — allows Lambda to write execution logs for every run



Screenshot 16 — IAM role before policies attached (1 policy only)



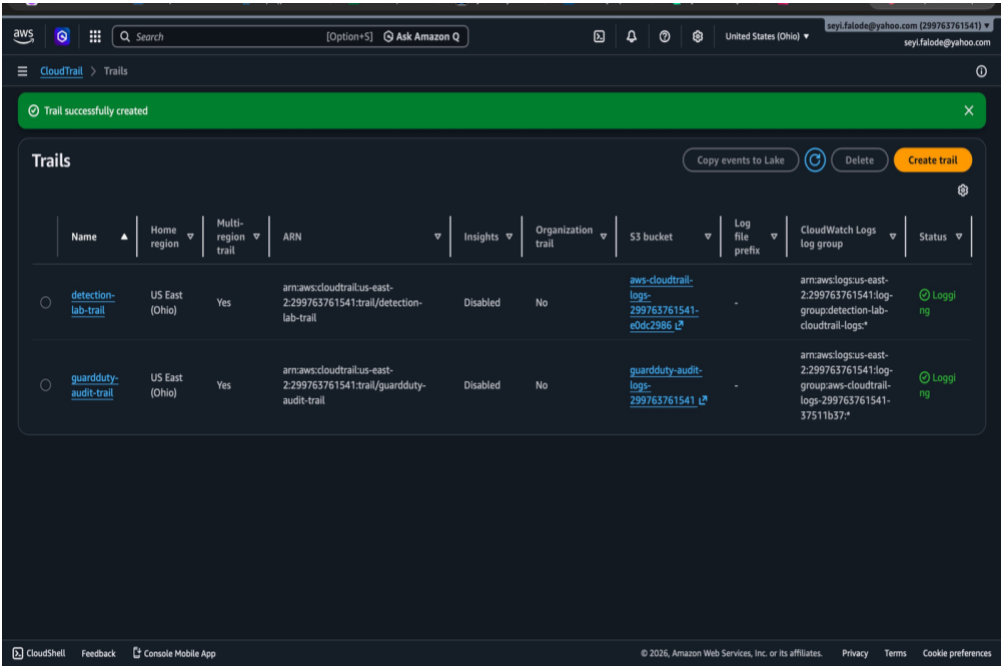
Screenshot 17 — All four policies successfully attached

PHASE 6 & 7: CLOUDTRAIL + S3 — AUDIT TRAIL & EVIDENCE STORAGE

CloudTrail captures every API call made in the account. When Lambda auto-remediates a finding — tagging an EC2 instance, modifying security groups — CloudTrail records the exact calls made, by which role, from which IP, at what time. This is the forensic evidence chain.

Trail Configuration

- Trail name: guardduty-audit-trail | Region: us-east-2 | Multi-region: Yes
- S3 bucket: guardduty-audit-logs-299763761541 — auto-provisioned during trail setup
- CloudWatch Logs integration: Enabled — real-time alerting on specific API patterns
- Log file validation: Enabled — detects tampering with stored log files
- Status: Logging



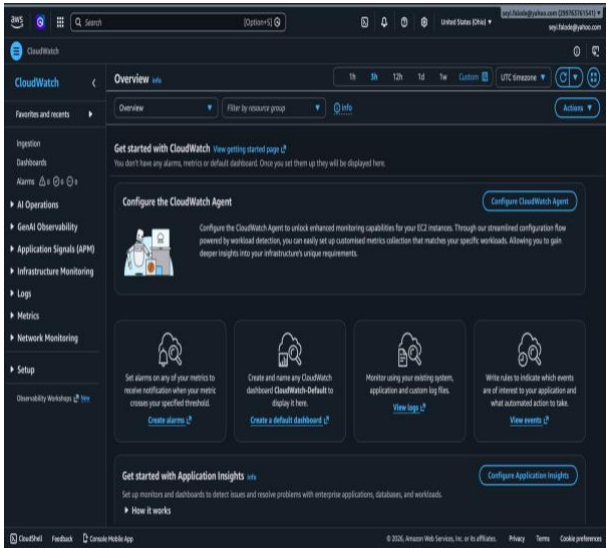
Screenshot 18 — CloudTrail guardduty-audit-trail created and actively logging

PHASE 8: CLOUDWATCH — MONITORING LAYER

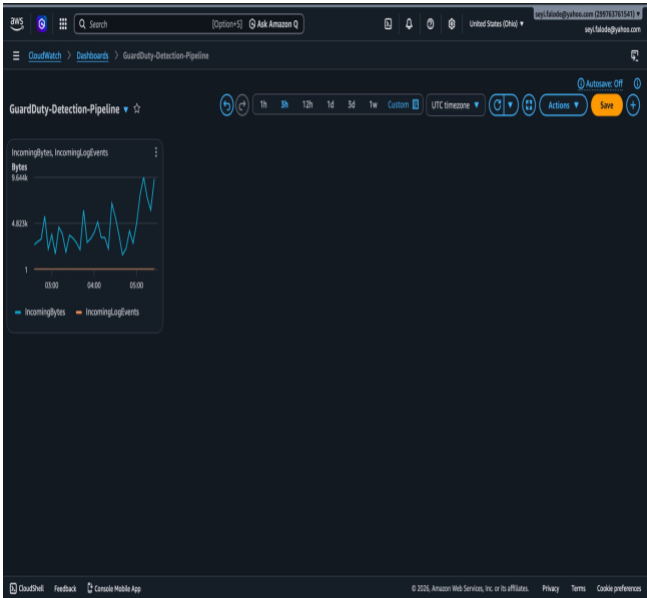
A CloudWatch dashboard named GuardDuty-Detection-Pipeline provides real-time visibility into pipeline health. This is how you verify the pipeline is working, identify gaps in log delivery, and detect if any component stops functioning.

Dashboard Metrics

- IncomingBytes — volume of log data flowing into CloudTrail log group
- IncomingLogEvents — number of individual log events being ingested per minute
- Log groups monitored: aws-cloudtrail-logs-299763761541-37511b37 and detection-lab-cloudtrail-logs

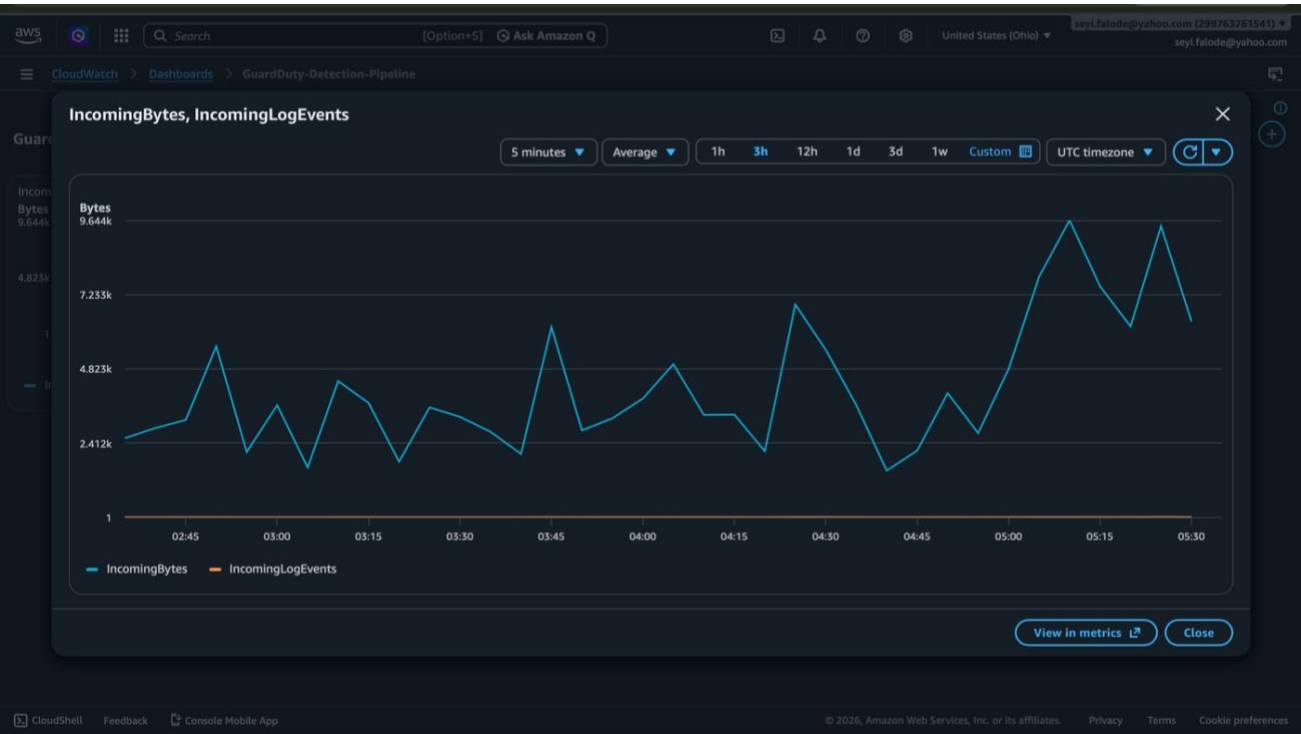


Screenshot 19 — CloudWatch homepage



Screenshot 20 — GuardDuty-Detection-Pipeline dashboard

Screenshot 21 — Live graph showing CloudTrail IncomingBytes activity in real time



END-TO-END PIPELINE TEST

With all eight components wired together, the pipeline was tested by generating sample GuardDuty findings and observing the complete chain from detection to email delivery.

Test Method

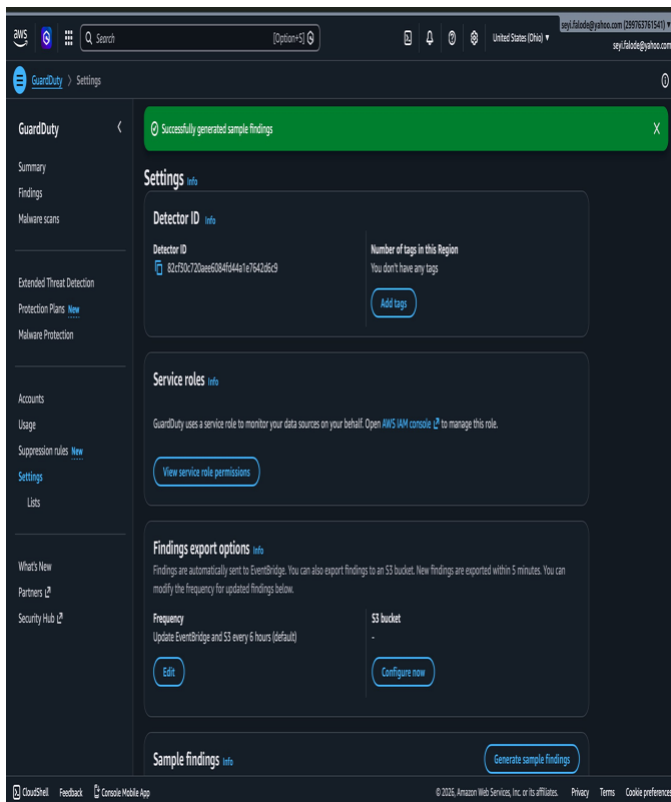
- Navigated to GuardDuty Settings → clicked Generate sample findings
- GuardDuty creates one sample finding for every supported finding type across all severities
- Observed findings flow through EventBridge → Lambda + SNS simultaneously

Results

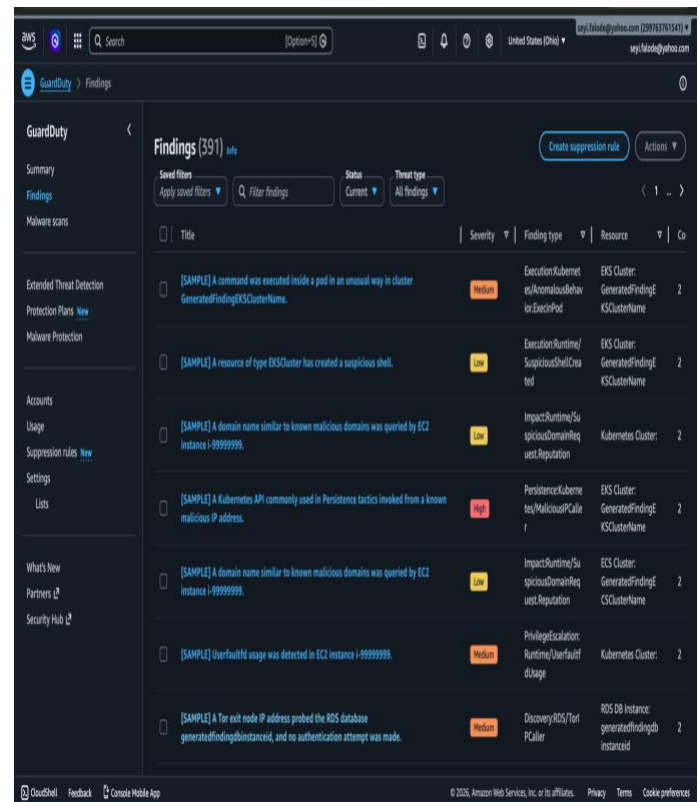
- 391 total sample findings generated across all severity levels and finding types
- EventBridge fired for all MEDIUM and HIGH findings (severity ≥ 4)
- 8 SNS alert emails received at seyi.falode@yahoo.com within seconds of generation
- Full JSON finding payloads delivered including all resource details, timestamps, MITRE mappings
- Lambda invoked for HIGH severity findings — execution logs confirmed in CloudWatch

Finding Types Observed in Alerts

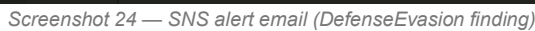
- DefenseEvasion:Kubernetes/SuccessfulAnonymousAccess
- DefenseEvasion:Runtime/FilelessExecution
- PrivilegeEscalation:Runtime/ContainerMountsHostDirectory
- Trojan:Runtime/BlackholeDNSTraffic
- Persistence:Runtime/SuspiciousCommand
- Execution:ECSCluster/MaliciousFile (EICAR test file detected)



Screenshot 22 — Sample findings generated successfully



Screenshot 23 — 391 findings including HIGH and MEDIUM severity



SKILLS DEMONSTRATED

Detection Engineering

- Designed complete threat detection lifecycle from scratch with no pre-built stacks
- Implemented severity-based triage logic to separate signal from noise and prevent alert fatigue
- Applied MITRE ATT&CK framework context to GuardDuty finding types
- Tuned automated response thresholds to prevent false positive disruption of workloads

Cloud Security (AWS)

- GuardDuty threat intelligence, finding triage, and sample findings generation
- EventBridge event-driven automation with JSON event pattern severity filtering
- Lambda serverless function development with boto3 SDK and Python 3.12
- SNS pub/sub alerting architecture with confirmed email subscription
- IAM least privilege role and policy configuration for cross-service access
- CloudTrail audit logging with CloudWatch Logs integration and log file validation
- S3 forensic evidence storage provisioned through CloudTrail trail setup

Security Automation & SOAR

- Automated EC2 isolation on HIGH severity findings with zero manual intervention
- End-to-end pipeline from detection to remediation executing in seconds
- Structured logging for every automated action via CloudWatch Logs

Incident Response

- Built complete forensic evidence chain: CloudTrail → S3 → CloudWatch
- Implemented tamper-evident log file validation on CloudTrail
- Isolation workflow preserves instance state for forensic analysis rather than terminating